INKA-2026 — Complete Work Specification

Statement of work for turning INKA-2026 into a genuine, complete and fully working games platform

Builder handoff document (AI Studio) · Issued 29 October 2026

Live site: https://inka-2026.vercel.app/ · **Repository:** https://github.com/eddyizere24-collab/Inka-2026

Primary objective: build everything that is missing, verify every part, and deliver a platform that real players can test. Audit the current site before changing anything. Nothing counts as finished simply because it appears on screen.

How to Use This Document

This is a work order, not a general idea or a wish list of nice-to-have features. The builder must inspect the live site, understand how it works, and then implement every requirement below. Do not stop after listing problems or presenting a plan.

First rule. Open the live site and the repository. Go through every page, every button, every game and every mobile interaction, and verify what you see against the code. After the audit, make real changes, test them, deploy them to the live site, and provide evidence that they work.

Status of every item. Every item in this document is a requirement. If something is genuinely impossible because of missing permissions, APIs or tools, say so explicitly: name the blocker and state what is needed to resolve it. Never add fake content to make something look finished.

If time is limited, work in this order: (1) real accounts and persistent data; (2) correct, tested game engines; (3) real online multiplayer; (4) honest data on every page, with no invented users, matches or scores; (5) brand identity and replacement of letter placeholders; (6) all remaining platform features. This ordering does not drop any requirement: anything left unfinished must appear in the final report as an open item, with the reason.

Key terms

- **Igisoro** — the traditional Rwandan mancala-style (seed-sowing) board game.
- **Dame** — draughts/checkers.

- **Biyari** — billiards/pool.
- **Real player** — a human with an authenticated account. An AI opponent is never presented as a real player.
- **Live** — a match that is genuinely in progress, as confirmed by the server.
- **Done** — an item that meets the criteria in Section 24 and has recorded evidence.

1. Purpose and Builder Obligations

1.1 What we are trying to achieve

Build INKA-2026 into a genuine, polished and reliable games platform that works on phones and computers, connects people who want to play, stores their accounts and history, and offers games that enforce real rules. It must not look like a temporary prototype or a page full of buttons that do nothing.

The seven pillars are:

1. Games that genuinely work.
2. A professional look and a distinctive INKA brand.
3. Real accounts and real data, stored securely.
4. Working play between people in different places.
5. History, points and rankings that come from matches that actually happened.
6. Every page serves a purpose.
7. Every claim of completion is backed by tests and by the live site.

1.2 Non-negotiable rules

- Do not change code until you understand the whole site as it currently works.
- Do not stop at analysis or planning. Change the code, test it and deploy it.
- Do not treat a demo, mock, placeholder, sample data or a game driven by random choices as finished work.
- Do not put invented numbers, names, live matches, comments or rewards on the public site as though they were real.
- Do not use plain letters such as “A”, “B”, “X” or “O”, or words written inside boxes, as game pieces or icons. Use purpose-made symbols and graphics.
- Do not remove working features before checking whether they are used elsewhere. Plan changes so that nothing that already works is broken.

- Do not put secrets, API keys or passwords in client-side code or in GitHub.
- Do not claim completion because the build passed. Every game and every core flow must be tested in practice.

1.3 What the builder must deliver

- The updated site, live at the main URL.
- Clean, maintainable code, with documentation of its structure.
- A database and server-side logic — or a precise description of the chosen technology and the reasons for choosing it.
- Tests proving that game rules, accounts, matchmaking, scoring and history work.
- A report listing what changed, what was tested, the results, what remains unfinished and why.
- A list of the tools or permissions required wherever an integration was impossible for lack of access.

2. Audit the Existing Site First — Do Not Build on Assumptions

2.1 Where to look

Where	What to do
Live site	Open https://inka-2026.vercel.app/ on a phone and on a computer. Visit every page, press every button, switch languages, start games, open profiles, and check the spectator, schedule and history areas.
GitHub	Read README.md, README.rw.md, README.fr.md, package.json, vercel.json, .github/workflows, the tests, and every file under src. Understand the dependencies, scripts, routes, state management and where data comes from.
Games	Open each game. Make legal and illegal moves; try winning, losing, drawing, restarting, leaving and returning. Check whether the opponent is a real person or a simulation.

Where	What to do
Mobile layout	Test small screens, menus, boards, buttons, dialogs, scrolling, portrait and landscape, and where the on-screen keyboard opens.
Network and data	Determine whether data comes from a backend or from front-end/local state. Check whether data disappears after a refresh or in a different browser or session.
Build and tests	Run the existing type-check, test, lint and build scripts. Record every error before changing anything.

2.2 Earlier findings to re-verify

An earlier code review found that the site behaved like a front-end prototype. It reported that some player and match data is written directly into the code; that some features run on local state only; that some functions merely simulate a challenge; that the games had no real server to connect players; and that the README says Biyari and its rules and physics still need testing. Confirm each of these on the current version before treating it as fact.

Verify and, where confirmed, fix the following:

- Current user, points, history, communityPlayers and liveMatches must not remain hard-coded data that looks real.
- Player messaging, challenges, comments and live matches need a genuine way to store and update data.
- Anything labelled “live”, “online”, “scheduled”, “ranked” or “real player” must reflect what is genuinely happening in the system.
- A game that offers random choices, or plays only against a simulation, must not be called a complete online two-player game.
- Every page in the menu must be fully functional. No page may exist for decoration only.

2.3 After the audit

Write a short summary of what you found, either in the work report or in the commit/PR. Do not wait for a reply: continue immediately and implement every requirement in this document.

3. INKA Brand and Visual Identity

3.1 Rule: no letter-based placeholder graphics

The site must look professional and make clear that INKA is a distinctive brand. We do not want large letters in boxes, circles marked “X/O”, random emoji, or temporary symbols standing in for cows, stones, game tokens or the logo. Anything that is meant to be a graphic must be a real graphic designed for this project.

- Create the INKA-2026 logo for use in the header, login screen, browser icon (favicon), loading screen, game pages and elsewhere. It must stay clear at every size.
- Create original game assets: seeds/markers for Igisoro, pieces for Dame, and balls and cue assets for Biyari. They must share one visual style that fits the brand without making the game hard to read.
- Where game elements show a cow or cow-like stones, make them purpose-designed, INKA-branded graphics. Letters must never stand in for a cow illustration.
- Use SVG or other high-quality scalable images. Store assets in a clear folder structure, add alt text where needed, and avoid duplicating heavy images unnecessarily.
- Define a design system — primary and secondary colours, typography, spacing, corner radius, shadows, buttons, cards, modals, tooltips, focus states and warning colours — so the whole site looks consistent.
- The logo must not be hard to read or larger than necessary, and must never be placed where it hides content or makes a game confusing.

3.2 Overall look and feel

- The homepage must clearly explain what INKA-2026 is, which games are available, how to sign in or create an account, and how to start playing.
- Use well-aligned layouts, enough spacing, readable text, clearly named buttons and understandable icons.
- Reduce clutter. Each page needs one main purpose, a clear message and a path to the next action.
- Use subtle animations that help people understand what happened. Avoid heavy, slow or distracting ones.
- Provide loading states, empty states, success messages and error messages that tell the user what to do next. Never leave a blank white screen or a spinner that never stops.

4. Pages and Their Functionality

Review the pages that already exist and refine them, but the final site must provide all of the functionality below. Page names may change for clarity; functionality may not be lost.

Page / area	Required functionality
Home	Logo; short description; the three games; a “Start playing” action; genuinely live matches, if any; how sign-in and sign-up work; a short introduction for new players.
Lobby / Find a game	Choose a game and play mode; level or experience where possible; players who are genuinely available; send and receive challenges; use matchmaking.
Games	For each game: a working board or table, controls, rules, a marker of whose turn it is, time/turn status, resign, restart or rematch, and a way to leave without causing errors.
Spectator	A list of genuinely live matches; join a match as a viewer; see score and time; comments if permitted; fast, automatic updates.
Scheduling	Choose an opponent, game, date and time; accept or decline invitations; edit or cancel; timezone handling and reminder information.
Profiles	Avatar, display name, unique account identifier, favourite game, statistics based on real data, permitted match history, and editable settings.
Rankings and rewards	Rankings and points calculated from validated data; badges unlocked only when their conditions are met; filters by game and time period.
Community	Discover players, view public profiles, follow/friend or invite where approved, moderated comments, report and block.

Page / area	Required functionality
Rules and help	Genuine rules for each game, how to play, FAQ, how to submit a problem, and readable privacy policy and terms.
Admin	Review reports, user accounts, match abuse, community content, game status and basic user numbers — accessible to administrators only.

5. Accounts, Identity Verification and Player Profiles

5.1 Registration and sign-in

- Create accounts through a secure method suited to the chosen technology. Provide email or phone verification if used, password protection, logout and password reset.
- Store passwords securely, through a reputable authentication provider or proper hashing. Never store them in readable form and never place them in the front end.
- Each account has a permanent unique identifier. When a player changes their display name or avatar, their match history and points must remain linked to the same account.
- Defend against repeated mass registration, expired sessions, excessive login attempts and sign-in from untrusted devices.
- Build verification flows that match the provider. Never show a “verified” mark on someone who has not been verified.

5.2 Player profiles

- Players can choose a display name, avatar or photo, language, favourite game and other non-confidential details.
- The profile keeps the display name separate from the account identifier, which makes impersonation harder.
- Statistics — games played, won, lost, drawn, points and level — are calculated from genuine match data.
- A profile must never expose a person’s private data, credentials or exact location without a valid reason and proper permission.
- Define how an account is deactivated or deleted and how its data is handled under the privacy policy. Do not delete other players’ match history without explaining why.

6. Lobby, Invitations and Starting a Match

- The “Find a player” button must search for a genuinely online player or place the user in a real matchmaking queue. It must never pick someone from an invented list.
- Every challenge has a clear status: sent, awaiting response, accepted, declined, completed or expired. Both sides must always see the up-to-date status.
- Before accepting, players must see the opponent's name, the game type and the time format where relevant.
- Matchmaking uses sensible criteria — the chosen game, availability, and skill or rating once built. It must not show offline players as if they were about to join.
- Creating a match must be one reliable (atomic) operation, so that a player cannot enter two matches at once, moves are not repeated, and two matches are not created from one request.
- Every challenge can be declined, deleted or left to expire. Notifications reach the right person, and a network error must have a way to restore the correct state.
- An AI or computer opponent may exist as a separate, clearly labelled mode. It must never be presented as a real person.

7. Principles That Apply to Every Game

- Each game has clearly written rules that match how the game is actually played. No legal move may be rejected and no illegal move may be accepted.
- Game state needs a trusted authority. For online games the server validates every move; never rely on the front end alone.
- Every move carries an identifier or sequence number, so that reconnecting cannot create a duplicate move and both players see the same order of events.
- Where a game uses a clock, the server's time is authoritative, so that a player cannot gain time by manipulating the browser.
- Each game must support: start, turn changes, move history, pause/reconnect where needed, win/loss/draw, resignation, a timeout policy and a defined end of game.
- The AI must choose only legal moves and have a clearly labelled difficulty level. Random moves that ignore the rules are not acceptable.
- Boards must be clear on phones: hit areas large enough to tap without precision, forgiving touch interaction, and a board that stays within sensible dimensions.
- Show whose turn it is, the winner, the clock and the progress of the match clearly — without using plain letters in place of assets.

- Every game has a basic tutorial, a way to read the rules and, where possible, a way to see the legal options again.
- Keep each game engine separate from the UI: the rules are tested on their own, and the UI displays only the state confirmed by the engine.
- Every game must have unit tests for legal and illegal moves, captures, turn order, win/loss/draw and special rule cases.

8. Igisoro: From Prototype to Complete Game

Igisoro must not be a board that merely looks right, or computer moves driven by chance. It must follow one chosen set of rules, document those rules on the rules page, and apply them identically to both players — including in online play.

8.1 Board, pieces and interaction

- Use a genuine Igisoro board with clear rows and pits/cells, coloured and shaped so the player can easily see where they may move.
- Seeds must be designed assets with a consistent look. Never use letters or numbers alone as game pieces.
- Show clearly each player's pits, the number of seeds in each, where seeds travel, and the direction of sowing and capturing under the chosen rules.
- Tapping a pit must reveal the legal moves. An illegal move produces a message that does not obstruct the player.
- Show whose turn it is, the last move, captured seeds, the clock and how the game ended.

8.2 Rules and game logic

- State clearly which variant of Igisoro has been built. Do not mix different rule sets without explaining.
- Implement sowing, how seeds are distributed, how captures work, when a move is illegal, and when the game ends, all under the approved variant.
- Write tests for every edge case: no legal move remaining, very few seeds, captures, continuing moves, moves that lack the required conditions, seed counts and the end of the game.
- The AI must check the legal moves before choosing. It may never make an impossible move or change the board outside the rules.

- Provide a practice mode, player versus computer, and player versus player online. If same-device pass-and-play remains, label it as a separate mode.

Critical point — do not assume the rules. Identify the chosen variant, find an authoritative description of its rules, record them in the technical documentation and test them. If a difference between rule sets needs a decision from the project owner, raise it as one clear question rather than inventing an answer.

9. Dame: A Rule-Enforcing Game, Not a Decorative Board

Dame must be a complete game: pieces that move legally, captures, promotion to king, turns, a defined end of game and a way to play again. No piece may change identity or move because of a state bug.

9.1 Board and pieces

- Use the correct board for the chosen Dame variant. If it is international draughts or English/American checkers, say which, and follow its differences.
- The two sides need distinct colours or graphics with enough contrast, and must look like real game pieces. INKA tokens must be properly illustrated and branded — not X/O letters or temporary circles.
- Show selection, legal moves, the capture route, king status and whose turn it is. Piece animation must never cause the front end and the server to hold different boards.

9.2 Rules that must be tested

- Diagonal movement under the chosen ruleset, with illegal moves rejected.
- Capturing an opponent's piece, including mandatory capture if the variant requires it.
- Multiple captures within one move where the rules allow, with a clear rule for when the capture route ends.
- Reaching the promotion row to become a king, with the rules for how kings move and capture.
- Detecting a win, no legal moves left, draw conditions, and time expiry where a time control is used.
- Confirming that double-taps, refreshes, reconnects or two open sessions cannot produce two moves or an incorrect win count.

9.3 AI and online play

The AI must choose among legal moves and have distinguishable difficulty levels. At a minimum it must avoid illegal moves and play sensibly; at higher levels it may use a search algorithm. Online play must use server-validated game state, and both players must receive the same update.

10. Biyari: A Working Game With Physics and Rules

Biyari must not be pictures of balls pushed around by improvised behaviour or inconsistent collisions. It must be a testable game with clear aiming, collisions, pocketing, fouls and turn changes.

10.1 What the player sees and does

- A table with realistic proportions, rails, pockets and clearly distinguishable balls. Ball assets have a consistent look and colours that are visible on a phone.
- A cue or aiming marker, a guideline showing direction, power control, a way to release the shot and a way to adjust the angle.
- Mouse and touch input handled correctly. The power meter is easy to understand, and a shot is never fired twice because both a touch event and a click event triggered.
- Collisions, friction, movement, rebound, pocketing and slowdown behave consistently. A ball must never pass through another ball or through a wall.
- Turns, pocketed balls, fouls, ball-in-hand or penalties (if the ruleset requires them) and the winner are all counted correctly.
- Show status while balls are still moving. Do not let the other player shoot before the current shot is complete unless the rules allow it.

10.2 Rules and testing

- Choose one recognised pool ruleset (for example 8-ball, or another well-known variant) and document it. Do not mix the rules of several variants.
- Test ball placement, breaks, legal shots, pocketing, fouls, hitting the wrong ball first, reaching the end of the game and winning.
- Physics must behave identically for online players. In a two-player game the server (or an equally trustworthy mechanism) determines the outcome; never depend on simulations that differ in each browser.

- Use a fixed-timestep, deterministic simulation, and a frame rate and calculation method that keep the game consistent even on a low-powered phone. Keep game logic separate from rendering.
- Provide a practice mode and a simple AI that takes only legal shots, then build online multiplayer so that both players see the same shot.

11. Playing Against People in Other Places: Real Multiplayer

Connecting a player with another over the internet is one of the main things that separates a complete platform from a prototype. A front end alone, or local state, is not enough. Build a backend and realtime layer that stores game sessions and sends validated updates to both players.

- Choose a backend that suits the project — for example a reliable database/auth/realtime service, or a custom server. Document why it was chosen, the data structure, how secrets are stored, and how costs and usage limits are monitored.
- Create a match session containing: session ID, players, game type, status, turn, state version, move history, created/updated timestamps and the result.
- Submit moves through an authorised route. The server checks the user, match membership, turn order, move legality and that the match has not already ended.
- Recover after a lost connection: when a player returns, they receive the latest state, no move is counted twice, and nothing they had already done is lost.
- Detect disconnected, reconnecting, opponent offline, match abandoned, and no server activity. The timeout and forfeit policy must be clearly defined and applied consistently.
- Prevent race conditions: both players cannot move at once when it is one player's turn, and duplicate requests must not have an effect twice.
- Provide error handling and retry that never start a new game or silently erase history.
- Test with two sessions belonging to two different people, on separate browsers or devices. Local testing in one browser is not enough to prove that online play works.

12. Spectator Arena: Watching Live Matches

- The list must come from the backend and show only matches that are genuinely LIVE. If no match is in progress, show an empty state saying so. Never invent players or scores.

- Each item shows the game, the player names, the score or state, the start time and the status.
- Spectators receive updates in real time (or at a reliable regular interval) and can neither change the game state nor send moves.
- Comments or reactions, if provided, need permissions, rate limits, report/block and spam control.
- Verify that nobody can view a private match or information not intended for the public.

13. Scheduling Matches and Reminders

- Availability is stored on the backend for each user. It does not vanish after a refresh or when signing in on another device.
- A scheduled match includes an opponent, game, date and time, timezone, invitation message and status.
- An invitation supports accept, decline, reschedule and cancel. Both players see changes once they are saved.
- Prevent double booking and invitations with a time that has already passed. If the server uses UTC, tell users clearly which time is being displayed.
- Notifications must be stored and readable, must prevent duplicates and, where possible, let users choose email, push or in-app delivery.
- If email or push integration is unavailable, in-app notifications must still work and the limitation must be stated. Never show “sent” when nothing was sent.

14. Points, History, Rankings and Rewards

14.1 The data must be real

- A match is saved when it ends, with its players, game, time, final outcome, moves or summary, and the reason it ended.
- The winner, loser or draw is decided by the game engine and the backend. A player must never choose their own outcome.
- Honour points, ratings and badges are calculated under written rules. Do not start any account with points from a game that was never played.
- The leaderboard has filters by game and by time period where needed. Avoid rankings that change because of local state or random seeds.

- A player's history shows the matches they are allowed to see, and clearly marks unusual or unrewarded matches wherever a policy covers them.
- Badges unlock only when their condition is met. Do not pre-award them to make a profile look complete.

14.2 Fair play and score protection

Provide ways to detect abnormal activity: points gained through forged requests, matches that finish suspiciously fast, duplicated results, or a player trying to alter state in the browser. Scoring decisions must be based on what the server recorded and must be traceable.

15. Community, Comments and Messaging

- The community player list comes from the database and respects privacy settings, blocks and player status. It must not be an array of names in the source code presented as real people.
- Comments have an author, match or thread, timestamp and status. Users can report problems to a moderator, and harmful comments are reviewed.
- Treat spam, harassment, impersonation and bursts of messages as things that must be monitored. Rate limits and reminders of the community rules are required.
- Online/offline status uses a presence state with a clear timeout. Never show "online" when it cannot be confirmed.
- Where messaging is built, let users block others from challenging or messaging them.

16. Admin, Moderation and Support

- Build an admin dashboard separate from the player site, protected by role-based access control. An ordinary user must never reach admin functions simply by changing the URL.
- Admins can view reports, open them, queue them, resolve them, reject them or take a permitted action. Keep an audit log of significant actions.
- Provide user suspension/ban, message removal, match-report review, and a way to restore service after an incident.
- Build a Help/Support page where users can submit a problem, check its status, or see where help is provided.

- Never create an admin account with a password written in the front end, or leave shared credentials in a public README.

17. Security, Privacy and Data Protection

- Verify authentication and authorisation on every important endpoint. A user must not be able to edit another user's profile, match, score or comments without permission.
- Validate on both the server and the client: inputs, IDs, lengths, file uploads, game moves and bursts of requests.
- Use database access rules/policies, rate limiting, secure cookie and session handling, and CSRF/XSS protection where needed, with secrets handled properly.
- Write a privacy policy, terms of use and community rules that explain what data is stored, why it is stored, and how users can update their data or delete their account.
- Do not store passwords, secrets, private keys or API credentials in the repository or in the browser bundle. Keep secrets in the environment variables of the platform or server.
- Log without exposing users' private information. Explain how logs, backups and match data are stored, and for how long.
- Create a backup and restore plan for important data if a database is used, and actually test a restore rather than merely stating that backups exist.
- Handle errors without exposing stack traces, tokens or other internal details to users.

18. Languages, Accessibility and Mobile Use

18.1 Languages

- Kinyarwanda, English and French must have matching translation keys. No page may mix languages or leave keys such as “game.start.title” visible on screen.
- Switching language must change all on-screen text, including buttons, errors, dialogs, game instructions, notifications, empty states and accessibility names.
- Store the selected language in a clear way. If it persists between sessions, follow the user's preference. Changing language must never destroy the state of a game.

18.2 Phones, computers and different user needs

- Use responsive design for different screen sizes, including small phones, tablets and desktops. There must be no unnecessary horizontal overflow and no overlapping elements.
- Buttons and tap targets must be easy to use by touch (recommended minimum 44 × 44 px). A board must not require zooming every time.
- Support keyboard navigation where possible, with visible focus indicators, labels on controls and sufficient contrast (target: WCAG 2.2 AA).
- Never use colour alone to show whose turn it is, an error or an outcome. Add a shape, icon or clear text label.
- Respect reduced-motion preferences: stop or soften animations for people who need that.
- Test visual quality on different browsers and on a slow network. Pages need practical loading, retry and offline/reconnect messages.

19. Technical Structure and Code Quality

- Continue using React/Vite/TypeScript where appropriate, but change the architecture wherever the prototype can no longer meet the requirements. Do not replace the whole framework without a practical reason.
- Separate UI components, game rules/engines, API/realtime services, database models, authentication, domain logic, translations and tests.
- Create a clear data model: users, profiles, matches, players per match, moves/events, match results, schedules/invitations, ratings/rewards, notifications, reports and audit logs, where needed.
- Use migrations or schema versioning so that database changes are repeatable and controlled.
- Use TypeScript types for API responses and game state, handle errors properly, and avoid ignoring types or using any everywhere.
- Review dependencies and unused packages, scripts, build warnings and bundle size. Do not add a large package only for decoration.
- Add an error boundary or equivalent, plus logging, monitoring and error tracking, so that you can see where the site fails.
- Document environment variables in .env.example without secrets. Production configuration must be separate from development and test.

- Use reusable components, design tokens and consistent naming. Do not create many pages with separate copies of the same code for no reason.

20. Performance, Stability and Reliability

- The homepage and games must open in a reasonable time on a normal connection. Use appropriately sized assets and lazy loading where it helps. Recommended target: usable on a mid-range phone on a typical mobile connection, with the main content visible in about 2.5 seconds.
- A game must not freeze while animating, or use so much CPU that an ordinary phone cannot play it.
- Use retry with backoff for network operations, while avoiding repeating a transaction or a move twice.
- Handle server errors, database unavailability, realtime disconnects, expired authentication and other failures. Messages must give the user a way to try again.
- Add health or status checks, or monitoring, suited to the chosen infrastructure. Know how to inspect the logs when a player reports that a match disappeared.
- An error in one game must not crash the whole page or display an unclear error message.

21. Required Testing — Looking Right Is Not Enough

21.1 Logic tests

- Unit tests for the rules of Igisoro, Dame and Biyari: legal and illegal moves, captures and fouls, turns, draw/win/loss and end states.
- Unit tests for scoring, badges, ratings, history, timeouts and match status.
- Tests for translation keys in Kinyarwanda, English and French, and for any place where untranslated text could remain.
- Tests for data validation, permission rules and API errors.

21.2 Integration and end-to-end tests

- Create an account, sign in, sign out, reset a password, change the profile, refresh the page and confirm that the data persists.

- Player A invites Player B; B accepts; the match starts; A makes a move; B sees it; B replies; the match ends; the outcome is saved in history.
- Test on two different browsers or devices, including a network disconnect, a reconnect and returning to the match.
- Test that a player cannot move on another player's turn, cannot join a match they were not invited to, and cannot change a score with a forged request.
- Test scheduling, accept/decline, reschedule, cancel and notifications.
- Test the spectator view: a LIVE match appears, a finished match leaves the live list, and a viewer cannot change the board.
- Test on a phone: menus, board, scrolling, forms, dialogs, game controls, language and layout in portrait and, where needed, landscape.

21.3 Technical and quality tests

- Run npm run build, npm test, npm run lint and any other existing scripts. Fix every failure and never hide one.
- Add end-to-end/browser tests for the key flows. Unit tests alone do not prove that the site works on screen.
- Test empty states, loading, errors, zero players online, an empty match list, match timeouts and many simultaneous users.
- Run a sanity (smoke) test on the production URL after deployment, not only on a local development server.
- Keep the test results and the screenshots or recordings that show the key things that were verified.

22. Publishing the Site and Handing Over the Work

- Use deployment previews for testing, then deploy to production after the checks pass. Do not publish untested changes.
- Verify that https://inka-2026.vercel.app/ opens for the intended audience, with no login wall or SSO unexpectedly protecting it.
- Check all environment variables, the backend URL, authentication redirects, allowed origins, database rules and other production configuration.
- After publishing, open the real URL in a browser and test the main routes, accounts and games. A deployment status of “READY” alone does not prove that all functionality works.

- Write a new or updated README: how to start the project locally, required environment variables, how to run the tests, architecture, database schema and migrations, deployment, and known unfinished items.
- Never put admin credentials or sample secrets in documentation. If test accounts are needed, use purpose-made test accounts that contain no personal data.
- Provide a commit or pull request, or a list of changes, showing what was done; the production link; the test results; and what the project owner must verify.

23. Work Phases — Do Not Skip Any

Phases may overlap, but none may be skipped, and a phase is not complete until its visible outcome can be demonstrated.

Phase	What to do	What must be visible
0. Audit	Review the site, GitHub, routes, components, tests, data flows and dependencies.	A report of findings and a list of confirmed problems.
1. Design / brand	Establish the INKA identity, game assets, design system and responsive navigation.	Clean-looking pages, no letter placeholders, a consistent UI.
2. Foundation	Authentication, database model, permissions, API/services, translations and error handling.	Accounts and data are stored and read securely.
3. Game engines	Complete and test the rules of Igisoro, Dame and Biyari.	Every game is playable, with legal moves, outcomes and tests.
4. Online play	Lobby, challenges, matchmaking, realtime state, reconnect and timeout.	Two people on separate devices can play the same match.
5. Platform features	Spectator, scheduling, history, rankings, badges, community, notifications.	Every page uses real data and every button has a purpose.
6. Security / QA	Permissions, abuse controls, end-to-end tests, mobile QA, performance, backups and monitoring.	Written tests; serious defects resolved.

Phase	What to do	What must be visible
7. Deploy / review	Production deployment, smoke tests, changelog, handoff.	A tested URL and evidence confirming each item.

24. Final Completion Checklist

The builder may not state that the work is finished until every item below has been checked and has a recorded result. This checklist will be used in the review of the work.

24.1 Look and use

- The site has the original INKA-2026 logo, used consistently.
- No letters are used as substitutes for icons, tokens or cows. Game assets are purpose-designed and clear.
- The site works well on phone, tablet and desktop; menus, boards, forms and modals do not break.
- All three languages work on every page, with no text keys or temporary wording left behind.
- Every page in the navigation is fully functional, or clearly explains when a service is unavailable.

24.2 Games

- Igisoro: the rules are confirmed and tested, the seeds and board are clearly visible, the AI makes no illegal moves, and online mode works.
- Dame: legal moves, captures, kings, end conditions and the ruleset all work and have tests.
- Biyari: aiming, power, collisions, pockets, turns, fouls and the end of the game all work and are tested.
- Each game can be started, played, finished, saved to history and played again.
- Online play has been tested by two people or accounts on two separate devices. Reconnecting, and a player leaving, have been tested.

24.3 Data and general functionality

- Accounts, profiles and preferences are saved. Sample data does not appear as the data of real users.
- Matches, scores, history, ratings and badges derive from real actions and are validated by the server.
- The spectator view shows only real live games. When no game is live, it shows an empty state.
- Scheduling, invitations, notifications, community and moderation all have tested flows.
- The admin area is protected by permissions. Ordinary users cannot change data that is not theirs.

24.4 Quality, security and delivery

- Build, tests, lint and type checks pass. Every failure is explained and none is hidden.
- API keys and secrets are not in client-visible code or in GitHub.
- Error states, loading states, empty states and network failures all have understandable responses.
- The production URL has been tested after deployment, and the key links open.
- README/documentation, schema and migrations, an environment configuration guide and a changelog are provided.
- The closing report lists the tests performed, what was used to run them, the results, the links, and anything that remains unfinished.

25. Owner's Review and Acceptance

Builder: be aware that a genuine review will follow.

After the site is updated, the project owner will open the production site, go through every page, play the games, test accounts and their functions, check that letters no longer stand in for graphics, and test an online game on two devices or two separate sessions. A good-looking screenshot, code that has not passed testing, or the word “done” will not be accepted as sufficient proof. What is said to work must actually work.

Before handing over the work as finished, submit the following:

- The production URL, and where all the pages can be found.
- A summary of the work done and the list of areas that were changed.
- The commit or pull request containing the changes.

- The results of the build, tests, lint and end-to-end tests.
- The list of tests performed on each game, including the key moves and how each outcome is verified.
- The results of the online-game test on two devices or sessions.
- Unresolved issues, the reason for each, and what is needed to resolve it. No issue may be hidden or passed off as completed work.

26. Conclusion: What You Must Not Forget

The goal is not to add decoration on top of a modest website. The goal is to turn a prototype into a games platform that genuinely works. That is why a game on screen must have real rules; a player on screen must be a real user, or an AI that is clearly labelled as AI; points must be calculated from games that were really played; “online” must truly mean online; and game pieces must be purpose-made assets, not letters in boxes.

First examine the live site, learn what is built and what is not, and then make the complete set of changes. Do not accept that a page opening means the system is finished. The site will be accepted only after real testing on production.

Your mission

Build, test, fix, publish and provide evidence. Do not answer this document with a summary of the work. Carry it out.

Starting points

- Live site: https://inka-2026.vercel.app/
- GitHub repository: https://github.com/eddyizere24-collab/Inka-2026
- Project name: INKA-2026

This document is the project's work order. The builder must confirm the earlier findings on the current version and then complete every requirement written here.

Appendix: Decisions the Project Owner Must Confirm

The items below need an owner decision. Where one is open, the builder proposes a documented default, continues with unrelated work, and flags the decision in the report as a single clear question.

1. **Igisoro variant** — which rule set, and which source, is authoritative.

2. **Dame ruleset** — international or English/American draughts, whether capture is mandatory, and how kings move.
3. **Biyari ruleset** — 8-ball is recommended unless the owner specifies another variant.
4. **Backend and authentication provider** — confirm the choice if the builder proposes changing the current stack.
5. **Points and ratings** — the honour-points and rating formula, and the list of badges with their unlock conditions.
6. **Timeout and forfeit policy** — turn time limits, and how long a disconnected player has before the match is forfeited.
7. **Notification channels** — which of in-app, email and push are available and within budget.
8. **Data retention** — how long logs, backups and match history are kept, and what happens to data when an account is deleted.
9. **Supported browsers and devices** — the minimum list to test against.
10. **Brand direction** — any existing colours, logo ideas or cultural and visual references the owner wants respected.